Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 9: **Data Processing: Import, Export, and Queues**

From the ETL pipeline, large datasets, and memory, to heavy work handed off to the background

What You'll Learn

1. Explain the **ETL** pattern and **batch processing**, and use them to process large amounts of data
2. Build a report that summarizes data over a date range, and pick the right techniques for **large datasets**
3. Understand why **memory** runs out, and choose memory-friendly ways to read data
4. Choose between **CSV, Excel, and PDF** for a need, and know the libraries that produce them
5. Recognize when a task should be handed off to a **queue**, and make it resilient to failure

> These slides cover the concepts. Applying reports, export/import, and queues to Simple POS is done in the lab jobsheet.

Meeting Map: Think of a Restaurant Kitchen



Picture a restaurant kitchen. Raw ingredients come in (**extract**), get cooked into dishes (**transform**), and are served at the table (**load**). When orders pile up, the kitchen does not make guests stand next to the stove: they get a table number, and the food is brought over later. That is the big picture of this meeting.

Part 1

ETL and Batch Processing

Four Basic Terms

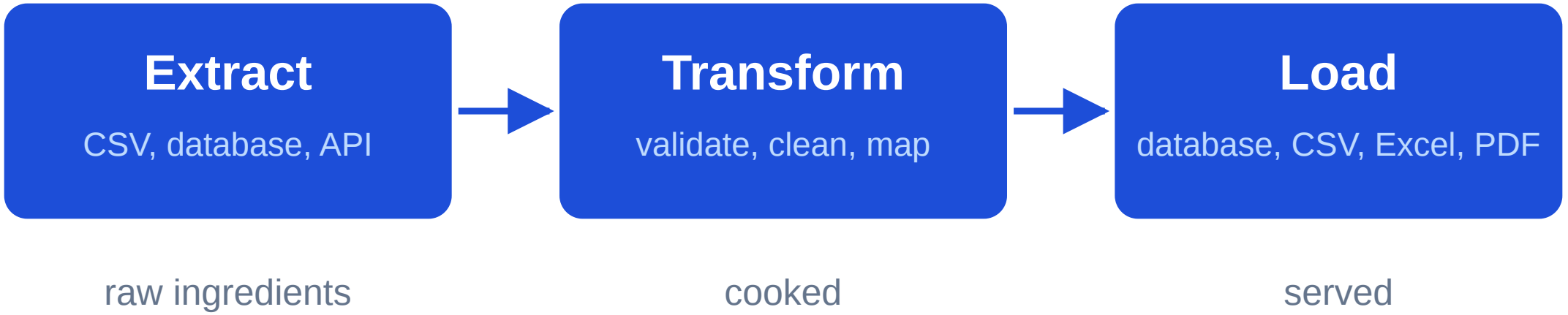
ETL (Extract, Transform, Load): a common pattern: take raw data, reshape it, then store or present it in a new form.

Batch Processing: processing lots of data together in groups, not one record at a time.

Queue: a mechanism for handing heavy work to a separate process that runs it in the background, instead of during the user's request.

Import/Export: bringing data in from an external format (CSV, Excel) into the database, or sending it out to an external format (CSV, Excel, PDF) to be read outside the app.

ETL: Three Stages That Always Show Up



Almost all data processing follows this order, for imports and exports alike. Separating the stages keeps each part easy to test and replace: the source can change, the validation rules can change, the destination can change.

Import and Export Are Two-Way ETL

	Extract	Transform	Load
Import	read a CSV / Excel file	validate each row, map columns	save to the database
Export	query the database	pick columns, format dates and numbers	write CSV / Excel / PDF

> Useful questions for every import or export feature: where does the data come from (extract), what must be cleaned or changed (transform), and where does the result go (load)?

Batch: One by One vs One Truck

A courier who delivers one parcel per trip loses to a courier who carries a full truckload. The same applies to databases: every query pays the cost of a trip (round trip) to the database.

```php
// 10,000 rows = 10,000 queries
foreach ($rows as $row) {
    Article::create($row);
}

// 10,000 rows = 20 queries (batches of 500 rows)
foreach (array_chunk($rows, 500) as $batch) {
    Article::insert($batch);
}
```

> A bulk `insert()` does not fill `created_at` / `updated_at` automatically and does not fire model events. Fill the timestamps yourself if you need them.

Batch Size and Reading in Batches

Batch too small

- Too many round trips
- Slow, like a motorbike courier

Batch too large

- Memory balloons
- One failure fails many rows

Reading in batches

```php
Article::where('status', 'draft')
    ->chunkById(500, function ($articles) {
        foreach ($articles as $a) {
            // process each row
        }
    });
```

> A reasonable starting point: hundreds to a thousand rows per batch, then measure. Wrap each batch in `DB::transaction()` if a batch must succeed or fail as a whole.

Idempotent: Safe to Run Many Times

What if the same import file is uploaded twice? Without a safeguard, every row is recorded twice. Think of an attendance sheet: if your name is already there, the clerk updates your line instead of writing the same name again.

```
Article::updateOrCreate(
    ['slug' => $row['slug']],     // lookup key
    ['title' => $row['title']],   // values to update
);

// batch version: one query for many rows
Article::upsert($batch, ['slug'], ['title']);
```

> `upsert()` requires a **unique index** on the key column. Without it, the database can't tell which rows count as the same.

Part 2

Reports and Large Datasets

List vs Report: A Pile of Receipts vs a Month-End Summary

A list page is like a pile of receipts in the cash drawer: everything is there, but you have to add it up yourself. A report page is like the month-end summary: already totaled and ready to read. The question is narrower but heavier: "what is the total of X over a given period".

```
$totalWords = Article::whereBetween('published_at', [
    $from, $to,
])->sum('word_count');
```

`whereBetween` keeps rows whose date falls between two bounds, then `sum()` adds up one column of the filtered result.

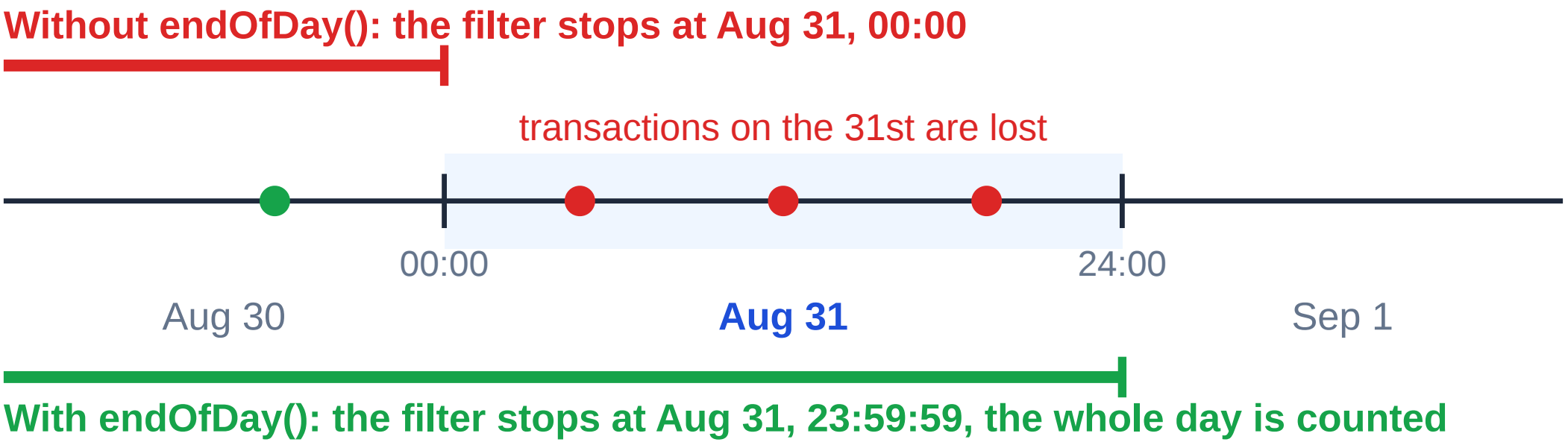
Common Pitfall: The Day That Gets Cut Off

> A filter of "August 1 to August 31" taken literally means "up to 00:00:00 on the 31st". Every transaction made during the 31st disappears, like a shop that tallies August revenue but stops counting the moment it opens its doors on the last day.

```
$from = $request->date('from')->startOfDay();
$to = $request->date('to')->endOfDay();
```

`startOfDay()` turns the start date into 00:00:00, and `endOfDay()` turns the end date into 23:59:59, so the whole last day is counted.

The Day-Boundary Pitfall on a Timeline



Calculate in the Database, Not in PHP

Imagine asking for your account balance. You could ask the bank to send every transaction and add them up yourself, or simply ask the bank for the balance. The database is the bank.

```
// heavy: every row is loaded into memory and summed in PHP
$total = Article::all()->sum('word_count');

// light: the database does the summing
$total = Article::sum('word_count');

// summary per author: the 5 authors with the most words
Article::selectRaw('author_id, COUNT(*) AS total, SUM(word_count) AS total_words')
    ->groupBy('author_id')
    ->orderByDesc('total_words')
    ->limit(5)
    ->get();
```

Stored Totals Are Not Recomputed

Meeting 6 made the point: a derived value like a total must not be trusted when it comes from client input, and must be recomputed on the server when the data is created. Once it is stored correctly, a report can use it as is. It is like the total printed on a receipt: when summarizing, you add up the receipt totals instead of re-checking every item on them.

```
// word_count is computed once when the article is saved
$article->word_count = str_word_count($article->body);

// the report just sums the stored column
$totalWords = Article::whereBetween('published_at', [$from, $to])
    ->sum('word_count');
```

> Recomputing a value from raw data (for example counting the words of every article body) each time the report opens burdens the server for no reason: the number has been correct since the article was saved.

Toolbox for Large Datasets

Technique	Use when	Notes
`paginate()`	showing a list to the user	one page per request
`chunkById()`	processing or updating many rows	memory per chunk, safe when rows change
`lazyById()`	reading many rows like a `foreach`	flat memory, simple code
`cursor()`	reading one long query	convenient, but less safe if data changes
`select([...])`	only some columns are needed	each model gets smaller
`with()`	using relations inside a loop	avoids N+1 queries
index	filtering or sorting on a column	query doesn't scan the whole table

Index: A Table of Contents for a Table

The date filter on the report runs again and again. Without an index on that column, the database has to read the whole table row by row, like finding one topic by flipping through the book page by page. With an index, the database jumps straight to the right page through the table of contents.

```
Schema::table('articles', function (Blueprint $table) {
    $table->index('published_at');
});
```

```
EXPLAIN SELECT * FROM articles WHERE published_at BETWEEN '2026-08-01' AND '2026-08-31';
-- type: ALL    = scans the whole table (slow)
-- type: range  = uses the index (fast)
```

> Recall Meeting 4: index strategy and `EXPLAIN`

Part 3

Memory Management

Why Memory Runs Out

PHP has a `memory_limit` (commonly 128 MB). Every row Eloquent loads becomes a full **model object** with attributes and metadata, much bigger than the raw data. Picture a workbench: `get()` puts every document on the bench at once. Once there are thousands of documents, the bench is full and work stops.

> Example measurement from the jobsheet: reading 52,500 transactions with `get()` uses about 96 MB, while `lazyById()` uses only about 38 MB. Add a bit more data and `get()` hits `memory_limit`: the error *Allowed memory size exhausted*.

`get` vs `chunk` vs `lazy` vs `cursor`

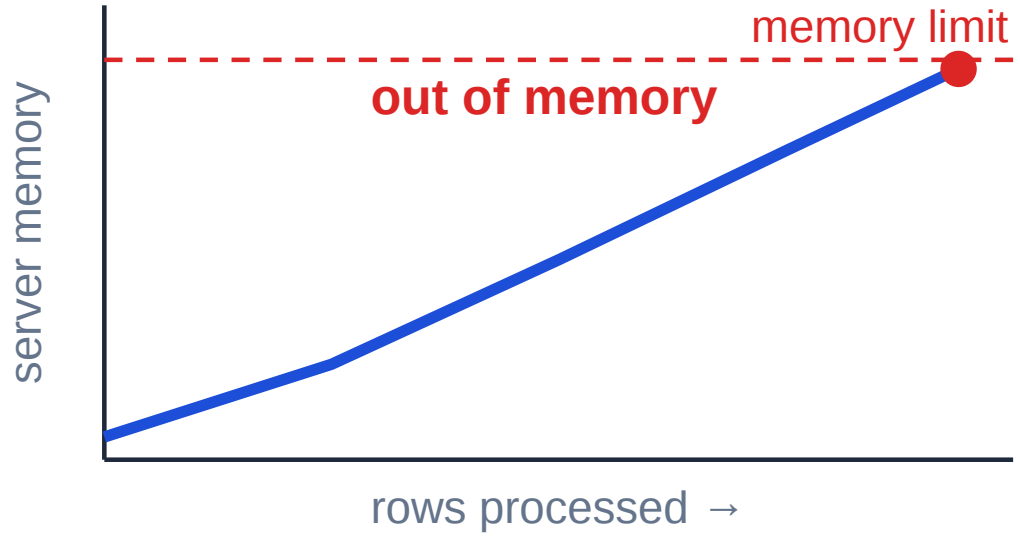
Way	Memory	Queries	Fits
`get()`	all rows at once	1	small data, shown on screen
`chunkById(n)`	n rows per turn	many (per chunk)	bulk updates, per-batch processing
`lazyById(n)`	n rows per turn	many (per chunk)	exports, a plain `foreach`
`cursor()`	one row per turn	1	a one-time read, data doesn't change

> Rule of thumb: if the data can reach hundreds of thousands of rows or more, don't `get()`. Use `lazyById()` to read and `chunkById()` to update.

Bucket vs Tap: Two Ways to Send a Big File

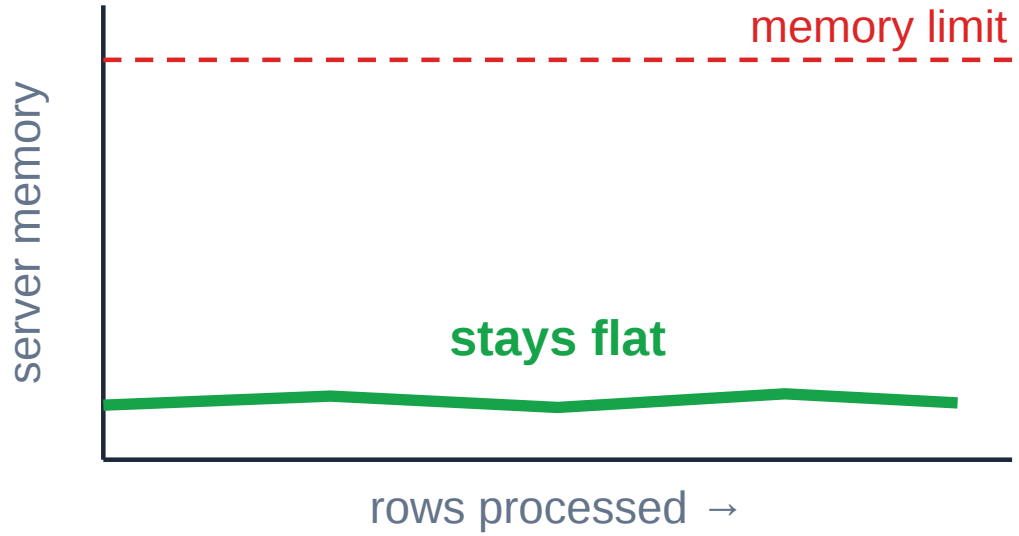
Bucket: collect first

get() then build one big string



Tap: flow row by row

lazyById() + streamDownload()



Export with `streamDownload()` and `lazyById()`

The tap way: fetch data in small chunks, write it to the output, then forget it.

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Title', 'Author', 'Date']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'articles-report.csv');
```

`lazyById()` reads the database in chunks, and `streamDownload()` sends each row as it goes, so memory stays flat.

> Laravel docs: `HTTP Responses → File Downloads` and `Eloquent → Chunking / lazyById()`

Don't Guess, Measure

```php
gc_collect_cycles();
memory_reset_peak_usage();      // restart the peak counter from zero

foreach (Article::lazyById() as $a) {
    // ... work ...
}

printf("peak %.1f MB\n", memory_get_peak_usage() / 1048576);
```

- `memory_get_peak_usage()` records the highest point of memory use
- `memory_reset_peak_usage()` (PHP 8.2+) makes each measurement stand on its own
- Compare two approaches on the same data, and write the numbers down

> Optimization without measurement is just a guess. Numbers before and after make the decision "use `lazyById()`" defensible.

The `chunk()` Trap: Skipped Rows

```php
// DANGEROUS: filters by 'draft', then changes 'draft' itself
Article::where('status', 'draft')->chunk(100, function ($articles) {
    foreach ($articles as $a) {
        $a->update(['status' => 'published']);
    }
});
```

Every updated row drops out of the filter, so the list shifts and the next chunk skips some rows. Like a ticket queue: when served people leave right away, everyone behind them moves up a number.

> The fix is `chunkById(100, ...)`: it uses the last `id` as the page marker instead of a position, so it is safe even when rows change.

Part 4

Formats and Libraries: CSV, Excel, PDF

Choosing a Format: CSV, Excel, or PDF

	CSV	Excel (.xlsx)	PDF
Best for	raw data, exchange between systems	analysis, financial reports	reading or printing, not processed again
Format	plain text	zip of XML, supports styling	fixed document
Memory	very light (stream)	medium (library handles chunks)	heavy (whole document in memory)
Library	built-in PHP (`fputcsv`)	Laravel Excel	DomPDF
Big data	yes	yes (use `FromQuery`)	no, limit the rows

CSV: Simple but Full of Traps

- **Delimiter**: Excel with an Indonesian locale often uses a semicolon (`;`) instead of a comma
- **Quoting**: values containing a comma or newline must be wrapped in quotes (`fputcsv` handles this)
- **Encoding**: UTF-8 files from Excel often start with an invisible character (**BOM**) that breaks the first header
- **No types**: every value is text, type validation is still your job

> If every row is suddenly rejected during an import, check the header first: the delimiter and the BOM are the prime suspects.

Laravel Excel: Export

A spreadsheet library handles the complicated `.xlsx` format. You only describe the content through small interfaces (`concerns`).

```
class ArticlesExport implements FromQuery, WithHeadings, WithMapping, ShouldAutoSize
{
    public function query()
    {
        return Article::with('author')->orderBy('id');
    }

    public function headings(): array { return ['Title', 'Author', 'Date']; }

    public function map($article): array
    {
        return [$article->title, $article->author->name, $article->published_at];
    }
}

return Excel::download(new ArticlesExport, 'articles.xlsx');
```

> docs.laravel-excel.com → Exports

Laravel Excel: Import

```php
class ArticlesImport implements ToModel, WithHeadingRow, WithValidation,
    WithChunkReading, WithBatchInserts
{
    public function model(array $row): Article
    {
        return new Article(['title' => $row['title'], 'author_id' => $row['author_id']]);
    }

    public function rules(): array
    {
        return ['title' => ['required'], 'author_id' => ['exists:authors,id']];
    }

    public function chunkSize(): int { return 1000; }   // read 1000 rows per turn
    public function batchSize(): int { return 500; }    // insert 500 rows per query
}

Excel::import(new ArticlesImport, $request->file('file'));
```

By default, `WithValidation` stops the import at the first invalid row. To skip bad rows and collect them, add `SkipsOnFailure`.

PDF with DomPDF: HTML Becomes a Document

Blade view → **HTML + CSS** → **DomPDF** → **PDF file**

```
$pdf = Pdf::loadView('reports.articles', ['articles' => $articles])
    ->setPaper('a4', 'landscape');   // wide table: use landscape

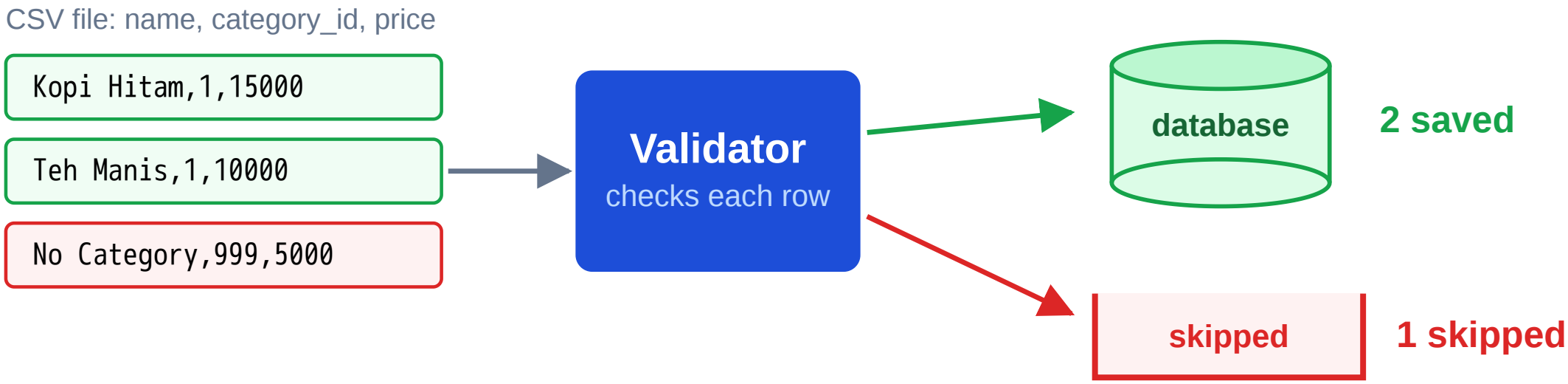
return $pdf->download('articles.pdf');
```

- The view is plain Blade, so `@foreach`, `{{ }}`, and basic CSS work right away
- `download()` forces a file download, `stream()` shows it in the browser

> DomPDF supports only part of CSS and holds the whole document in memory. Use it for summaries and small tables, not for tens of thousands of rows: for big data choose CSV or Excel.

Import: Like a Cashier Scanning Groceries

Import goes the other way: read a CSV file, then turn each of its rows into a new database row. Think of a cashier scanning groceries. If one barcode is damaged, the cashier sets that item aside and keeps scanning the rest, instead of sending every customer home.



one bad row does not stop the others

Import: Validate Each Row, Skip the Bad Ones

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $data = array_combine($header, $row);

    $validator = Validator::make($data, [
        'slug' => ['required', 'string'],
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // skip this row, move on to the next one
    }

    Article::updateOrCreate(['slug' => $data['slug']], $validator->validated());
}
```

What to Do with a Bad Row?

Strategy	How it works	Fits
Skip without a trace	bad rows are skipped with no record	almost never: data vanishes and nobody knows
Skip and log	bad rows are skipped, their row numbers and reasons saved	general imports (recommended): the user can fix the file
Abort everything	one bad row cancels the whole import (`DB::transaction`)	financial data or data that must stay whole

> Like the cashier earlier: the item with the damaged barcode is set aside, **and** noted on a "could not scan" slip so the customer knows what to sort out.

Part 5

Queues: Work in the Background

Two Ways to Process Work

Directly (inside the request)

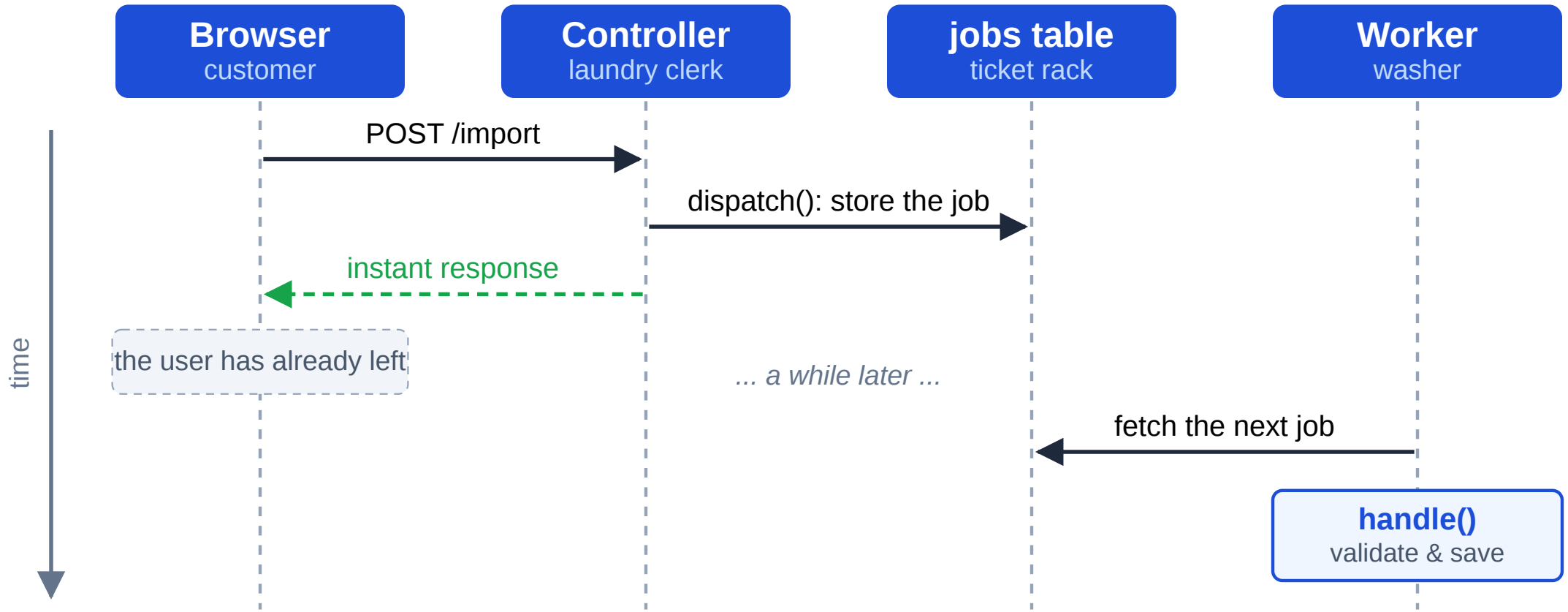
- The user waits until it finishes
- If it fails, the user knows right away
- Fits short tasks whose result is needed immediately

Via a queue (job)

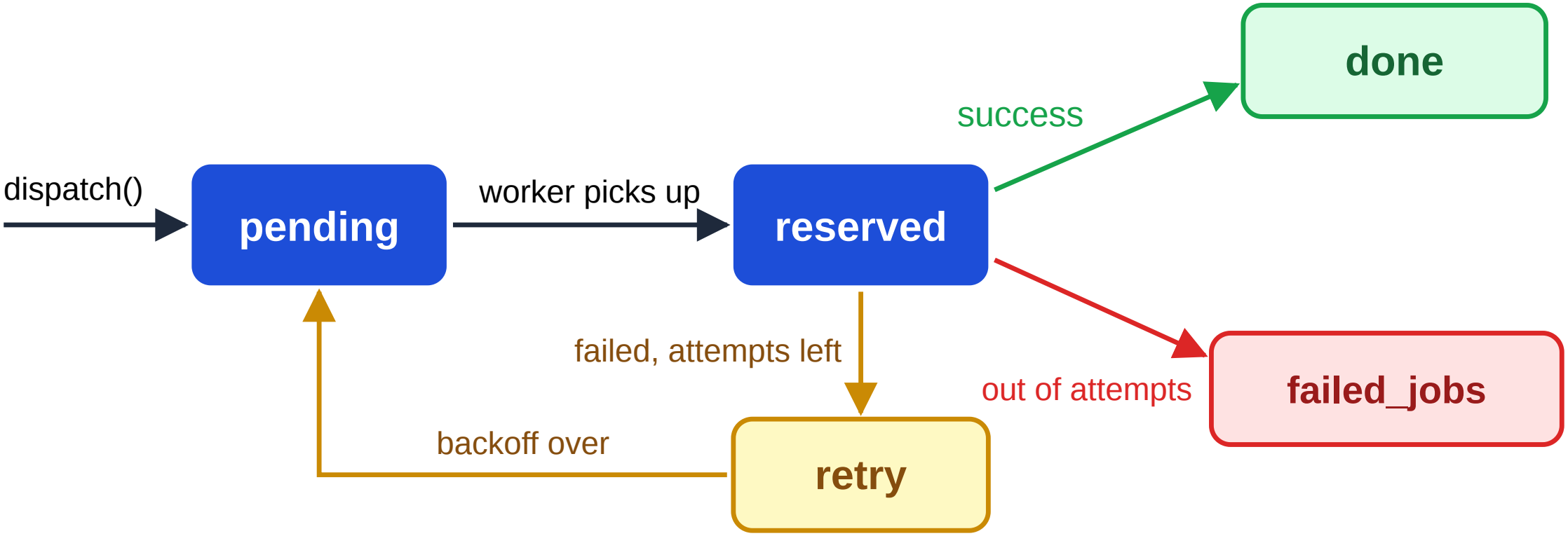
- Instant response, work runs in the background
- The user does not wait in front of the browser
- Fits heavy tasks or tasks with unpredictable duration

> Laundry analogy: one T-shirt, you gladly wait at the counter. A whole sack, you drop it off, get a ticket, and go home. Importing a few dozen rows is one T-shirt. Importing thousands of rows is a whole sack, and HTTP requests have a timeout.

A Queue = A Laundry Drop-Off



The Journey of a Job




```
$tries = 3, $backoff = [10, 60]
```

The Job Flow in Laravel

Controller → **dispatch() job** → **jobs table** → **Worker (queue:work)**

```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);

return back()->with('success', 'The import is being processed in the background.');
```

The controller only saves the file and sends one job to the queue, then responds to the user right away. The real work (the validation and saving loop) moves into the `handle()` method of that job class.

Worker: The Person Who Works Through the Ticket Pile

> **Worker:** a separate PHP process that keeps running, takes jobs from the `jobs` table one at a time, and runs each one's `handle()` method.

```
php artisan queue:work
```

Without a worker, jobs just pile up in the `jobs` table. It is a laundry with no one to wash: tickets keep growing, but no laundry gets done, because `dispatch()` only stores the ticket, it does not wash anything.

> A worker is a long-lived process, so it does **not** pick up new code automatically. After changing a job class, restart the worker (on a production server: `php artisan queue:restart`).

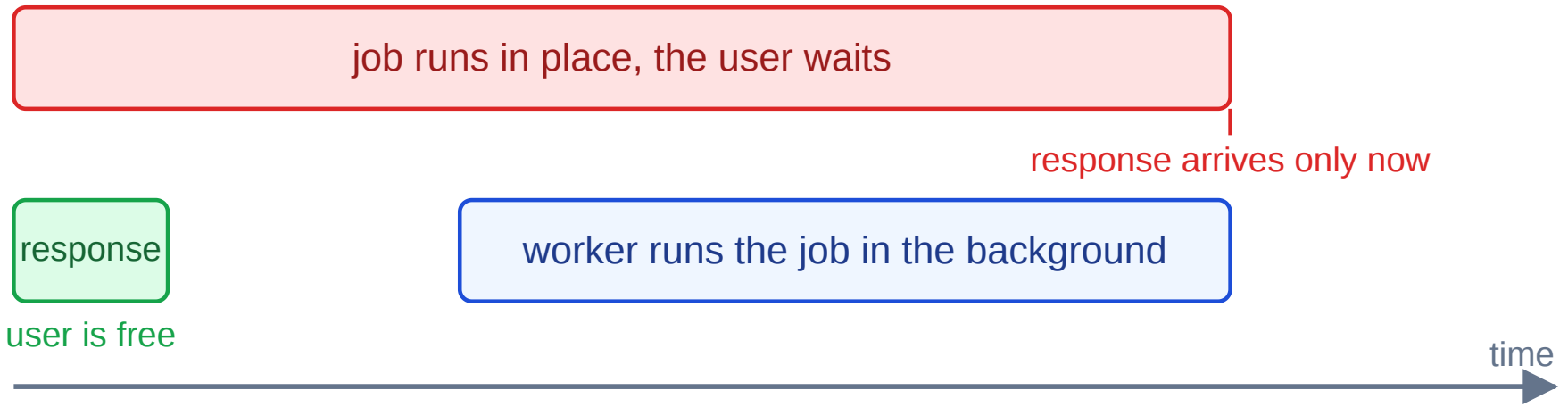
Queue Drivers and Workers in Production

Driver	Storage	Fits
`sync`	none: the job runs right there	development and testing
`database`	the `jobs` table	small to medium projects, easiest setup
`redis`	Redis (in-memory)	high traffic, high throughput needs

In production, `queue:work` must **stay alive**. A program like **Supervisor** watches the worker and starts it again when it dies, like a shift supervisor making sure the washer is always at their post.

> Laravel docs: `Queues → Supervisor Configuration`

`sync` vs `database`: Who Waits?



If `QUEUE_CONNECTION` in `.env` says `sync`, the job never enters a real queue. It is like a clerk telling you to wait until the laundry is dry even though you are already holding a ticket.

A Job That Survives Failure

A job runs in the background, so no user sees its errors. That is why a job must be designed to fail gracefully.

```
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3;                // at most 3 attempts
    public array $backoff = [10, 60];     // wait 10 seconds, then 60 seconds
    public int $timeout = 120;            // cancel if it runs longer than 120 seconds
}
```

```
php artisan queue:failed        # list failed jobs
php artisan queue:retry all     # run them all again
php artisan queue:flush         # delete the failed job records
```

> Laravel docs: `Queues → Dealing With Failed Jobs`

Rules for Designing a Job

Do

- Make jobs **idempotent**: safe to retry (`updateOrCreate`)
- Carry an ID or file path, not big data
- Set `$tries`, `$backoff`, and `$timeout`
- Use `->afterCommit()` when the job depends on data inside a database transaction

Avoid

- Using `request()`, the session, or the logged-in user inside `handle()`: in the background, none of them exist
- Storing a whole file's contents in the job payload
- Relying on job order without designing for it
- Letting jobs fail with nobody watching

Keeping the User Informed

A user who uploads a file and sees "being processed" wants to know when it is done. The options:

- **A status column** on the import table (`pending`, `processing`, `done`, `failed`) updated by the job and shown on the page
- **A notification** (email or an in-app notification) when the job finishes or fails
- **A job batch**: `Bus::batch($jobs)->dispatch()` tracks many jobs at once, and `$batch->progress()` gives the percentage done

> Laravel docs: `Queues → Job Batching`

Signs a Task Belongs in a Queue

- Its duration cannot be predicted in advance, since it depends on the size of the data
- The user does not need to see the result immediately on the same page
- It calls a slow external service, such as sending email or calling a third-party API
- It can be retried safely (idempotent)

> A report that is opened and read right on the page is usually still better done directly, because the user is waiting for the result at that moment. A queue is not the answer to every heavy task, only to those that do not need an instant result.

Applying It to Simple POS

You will apply this meeting's concepts directly to Simple POS in the lab jobsheet:

1. A sales report with a date filter, computed in the database
2. Export the report to CSV (streaming), Excel (Laravel Excel), and PDF (DomPDF)
3. Measure memory of `get()` vs `lazyById()` on 50,000 rows
4. A product import from CSV that validates per row and is safe to repeat
5. Move the import to a queued job with `$tries` and `backoff`, then trace the failed jobs

> Full code: `github.com/se-polinema/simple-pos-ch08`

Summary (1/3): ETL, Batch, and Large Datasets

- ETL splits data processing into extract, transform, load; import and export are two-way ETL
- Batching cuts database round trips (`array_chunk` + `insert`, `chunkById`); batch size should be measured
- A good import is idempotent: `updateOrCreate()` or `upsert()` (with a unique index)
- Reports summarize through `whereBetween`, and `startOfDay()` / `endOfDay()` keep the last day from being cut off
- Calculate in the database (`sum`, `groupBy`), not in PHP; an index speeds up filters

Summary (2/3): Memory and Formats

- Memory runs out because `get()` loads every row as an object; use `lazyById()` to read and `chunkById()` to update
- `chunk()` can skip rows when the filter column is also updated: use `chunkById()`
- Measure with `memory_get_peak_usage()`, don't guess
- CSV for raw data, Excel (Laravel Excel) for analysis, PDF (DomPDF) for compact documents
- Bad rows during an import: skip and log is the common choice, abort for data that must stay whole

Summary (3/3): Queues

- Work with unpredictable duration or large volume goes to a queue: instant response, work follows
- `dispatch()` only stores the job in the `jobs` table; the worker (`queue:work`) runs it
- Resilient jobs: `$tries`, `$backoff`, `$timeout`, and the `failed_jobs` table with `queue:retry`
- Design idempotent jobs, carry an ID or path (not big data), and don't depend on the request or session
- In production, Supervisor keeps the worker alive and it is restarted on deploy

References & Discussion

Official Laravel documentation (Queues, File Storage, HTTP Responses, Eloquent Chunking), docs.laravel-excel.com, and laravel-dompdf

Full code: `github.com/se-polinema/simple-pos-ch08`

Next meeting: Designing and Building a REST API